

Strings = Text o Cadenes

1- Tractament de cadenes

- 1.1. Variables de text
- 1.2. Operacions amb cadenes
- 1.3. Índex o posició d'una cadena
- 1.4. Mètodes o funcions de les cadenes
- 1.5. Algunes funcions de cadenes



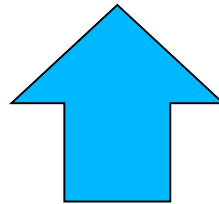
1.1. Variables de text o cadenes

- Les variables de text s'anomenen cadenes o **strings** (en anglès).
- Tenen el tipus **str**.
- Les cadenes en Python estan delimitades per **cometes** simples o dobles
 "hola" és el mateix que **'hola'**
- Podeu mostrar un literal de cadena amb la funció `print()`

```
print("Hola")  
print('Hola')
```

Crea el programa 02-strings.py

```
frase1 = "Les 'cometes' de dins són simples"  
frase2 = 'Les "cometes" de dins són dobles'  
frase3 = " Dobles \"cometes\" i a dins també dobles"  
print(frase1)  
print(frase2)  
print(frase3)
```



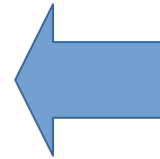
Marca les cometes amb una tecla especial

Un cop comprovat que funciona bé, comenta el codi per poder seguint provant coses al mateix programa

```
print(frase1)
print(frase2)
print(frase3)
```

Signes especials :

```
print ("33.4 \t\t 12.12 \t\t")
```



\t = tabulador

\n = salt de línia o newline

```
a = """Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."""
print(a)
```

Una cadena de tipus
paràgraf.

Les rutes de fitxers poden posar-se

- amb r davant de la cadena
- o bé amb doble barra
- o bé amb la barra /

```
print ("c:\test\scripts")
```

```
print (r"c:\test\scripts")
```

```
print ("c:\\test\\scripts")
```

```
print ("c:/test/scripts")
```

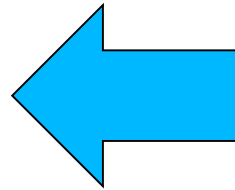
```
c:      est\scripts
c:\test\scripts
c:\test\scripts
c:/test/scripts
```

Exercici : Executa aquestes instruccions per entendre com funcionen

```
print ("Hola-1" "Adios")  
print ("Hola-2", "Adios")  
print ("Hola-3"); print("Adios")  
print ("Hola-4"); print(); print("Adios")  
print ("Hola-5\nAdios")
```

1.2. Operacions amb cadenes

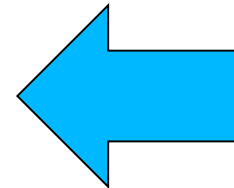
```
1 c1 = "ala"  
2 c2 = "pep"  
3  
4 print (c1+c2)
```



La suma concatena

alapep

```
1 print (c1 * 4 + c2)
```



Multiplicar
és repetir

alaalaalapep

1.3. Índex o posició d'una cadena

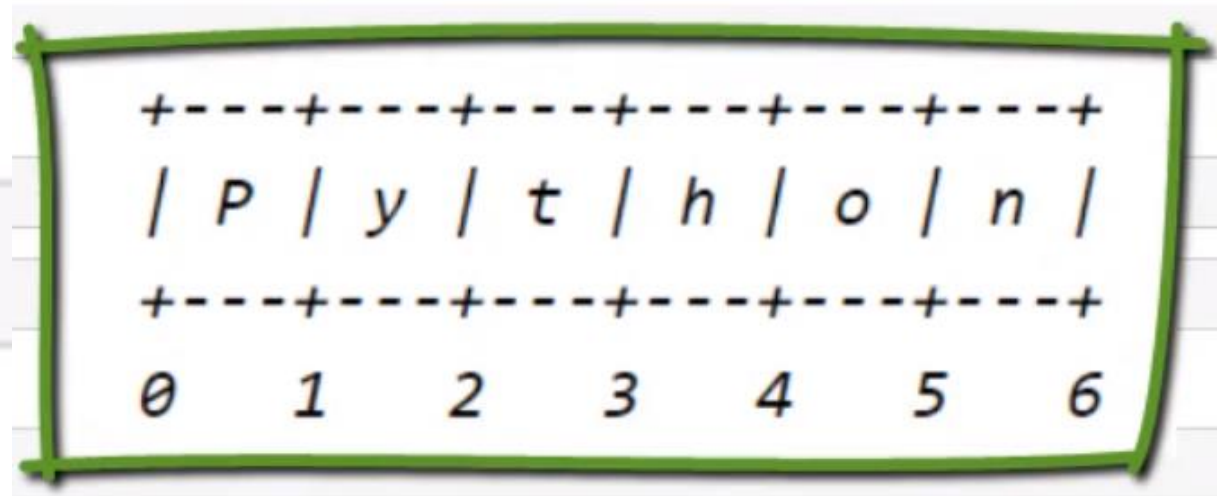
```
palabra = "Python"
```

```
palabra[0] # caràcter en la posició 0
```

'p'

```
palabra[3]
```

'h'



	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
		P		y		t		h		o		n				
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
	0		1		2		3		4		5		6			

Slicing o Llesques d'un text

```
In [47]: palabra[99]
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-47-b31ddef6ab27> in <module>()  
----> 1 palabra[99]  
  
IndexError: string index out of range
```

```
In [48]: palabra[:99]
```

```
Out[48]: 'Python'
```

```
+---+---+---+---+---+  
| P | y | t | h | o | n |
```

```
+---+---+---+---+---+
```

```
0    1    2    3    4    5
```

```
-6   -5   -4   -3   -2   -1
```

```
In [50]: palabra[99:]
```

```
Out[50]: ''
```

```
In [51]: palabra[0] = "N"
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-51-c87a9e773639> in <module>()  
----> 1 palabra[0] = "N"  
  
TypeError: 'str' object does not support item assignment
```

```
In [52]: palabra = "N" + palabra[1:]  
palabra
```

```
Out[52]: 'Nython'
```

```
In [31]: palabra[-0]
```

```
Out[31]: 'p'
```

```
In [32]: palabra[-1]
```

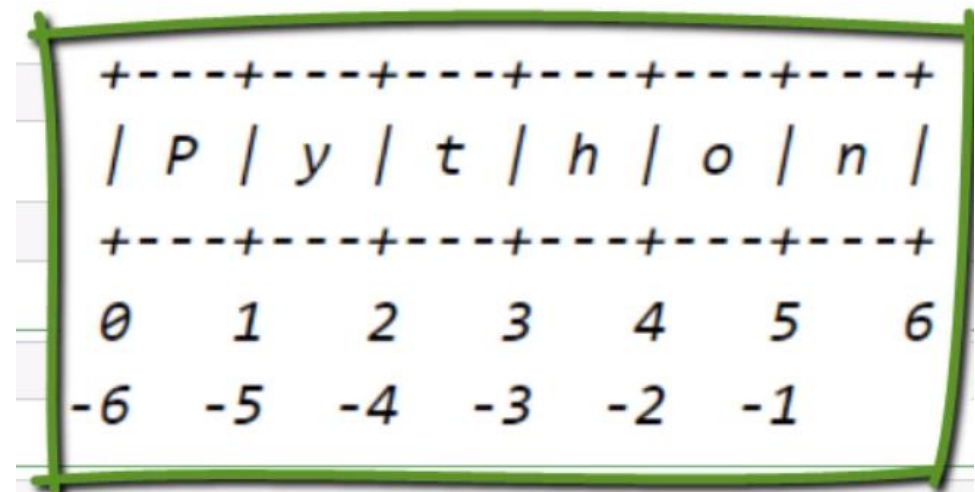
```
Out[32]: 'n'
```

```
In [33]: palabra[-2]
```

```
Out[33]: 'o'
```

```
In [35]: palabra[-6]
```

```
Out[35]: 'p'
```



+---+---+---+---+---+						
	p		y		t	
+---+---+---+---+---+						
0	1	2	3	4	5	6
-6	-5	-4	-3	-2	-1	

```
palabra = "Python"
```

```
palabra[0:2]
```

```
'Py'
```

```
palabra[2:]
```

```
'thon'
```

```
palabra[:2]
```

```
'Py'
```

```
palabra[:]
```

```
'Python'
```

```
+---+---+---+---+---+---+
| P | y | t | h | o | n |
+---+---+---+---+---+---+
0   1   2   3   4   5   6
```

P01. Pràctica de cadenes



str1="son"

str2="ma"

str3="ca"

str4="rro"

Tenim les cadenes str1 a str4.

Forma les següents paraules, operant les cadenes i sense afegir cap lletra més :

1. maca

4. roca

2. carro

5. caso

3. macarro

6. moco

P02. Més Llesques (Slicing) :

Tenint la variable text = "MOSCATELL" calcula mentalment i comprova :

text[1:4]

text[:5]

text[-4:]

text[2]

text[:]

text[::-1]

P03-1. Dates

Tenim la cadena data = “25/02/1985” en format **dd/mm/aaaa**

Usant slicing escriu-la de les següents maneres :

-aaaa/mm/dd

-mm/dd/aaaa

-dd **nom_mes** (*) aaaa

(*) *Agafa un nom del mes aleatori de Gener Maig.*

```
import random
mesos = ["Gener", "Febrer", "Març", "Abril", "Maig"]
nom_mes = random.choice (mesos)
```

P03-2. Avançat :

Utilitza el slicing amb aquesta **cadena** per generar una llista vertical de noms femenins.

Si agafes parts de paraules, agafa com a mínim dues lletres de cada paraula.

Quan els mostris, posa la inicial amb majúscula i la resta en minúscula.

cadena = ""L'advocació de Maria de la Mercè està lligada a la tasca carismàtica de l'ordre religiosa mercedària, nascuda a Barcelona amb sant Pere Nolasc i sant Ramon de Penyafort, amb el suport de Jaume I el Conqueridor""

Pots obtenir les posicions copiant el text al Notepad++

Soluciones posibles

```
text = "MOSCATELL"
```

```
#P02: calcula text[1:4], text[:5], text[-4:], text[2], text[:],  
#          text[::-1]
```

```
print(text[1:4])  
print(text[:5])  
print(text[-4:])  
print(text[2])  
print(text[:])  
print(text[::-1])
```

OSC
MOSCA
TELL
S
MOSCATELL
LLETACSOM

```
#P03-2: Slicing Mercè
```

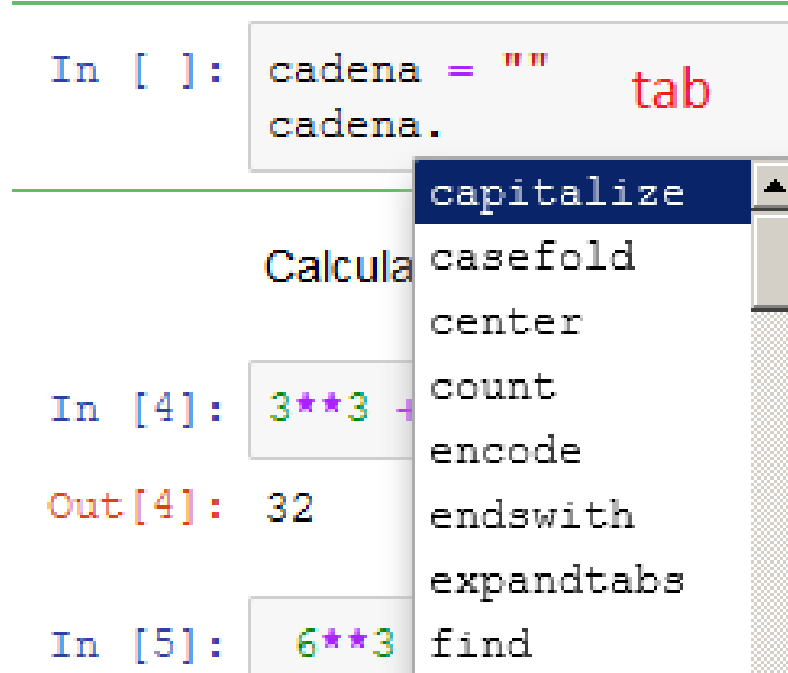
```
print (cadena[15:20])  
print (cadena[27:32])  
print (cadena[28:31])
```

Maria
Mercè
Ada
Ona
Daria
Ramona
Jana
Mati

1.4. Mètodes o funcions dels strings o cadenes de text

Quan tenim una variable de tipus **cadena**, tenim tot un seguit de funcions o mètodes que podem usar.

Els podem veure al Jupyter si creen una variable, i després posem el nom de la variable seguida d'un punt i prement la tecla **tabulador**.



The screenshot shows a Jupyter Notebook interface. The top cell, labeled 'In []:', contains the code `cadena = ""` followed by a red 'tab' label. Below this, a dropdown menu is open, displaying a list of string methods: `capitalize`, `casefold`, `center`, `count`, `encode`, `endswith`, `expandtabs`, and `find`. The `capitalize` method is currently selected and highlighted in blue. Below the dropdown, the text 'Calcula' is visible. The bottom part of the notebook shows two more cells: 'In [4]:' with the code `3**3` and 'Out[4]:' with the output `32`, followed by 'In [5]:' with the code `6**3`.

```
In [ ]: cadena = ""
cadena.

In [4]: 3**3
Out[4]: 32

In [5]: 6**3
```

- `capitalize`
- `casefold`
- `center`
- `count`
- `encode`
- `endswith`
- `expandtabs`
- `find`

Hi ha moltes funcions per a les cadenes.
Les anirem veient amb calma al llarg del curs.
Es poden dividir temàticament...

- Buscar, reemplaçar, comptar les vegades que apareix un caràcter, traduir

`find()`, `replace()`, `rfind()`, `index()`, `count()`, `maketrans()`, ..

- Canviar de tipus : ajuntant o separant cadenes

`join()`, `rpartition()`, `split()`, `rsplit()`, `splitlines()`, ...

- Formatar el contingut, posant traient espais

`format()`, `expandtabs()`, `center()`, `ljust()`, `rstrip()`, ...

1.5. Exemples de funcions de cadenes

<code>upper()</code>	Passa una cadena a majúscules
<code>lower()</code>	Passa una cadena a minúscules
<code>capitalize()</code>	Passa la primera lletra del text a Majúscules.
<code>title()</code>	Posa la primera lletra de cada paraula en majúscules

```
frase = "Python is Cool !"
```

```
print ( frase.upper() )  
print ( frase.lower() )  
print ( frase.capitalize() )
```



```
PYTHON IS COOL !  
python is cool !  
Python is cool !
```

`center ()`, `ljust ()` i `rjust ()` alineen una cadena al centre, a l'esquerra o la dreta respectivament

`isupper()` Returns True if all characters in the string are upper case

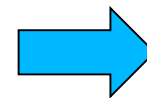
`islower()` “ “ “ all characters in the string are lower case

`istitle()` “ “ if each word starts with uppercase

`isnumeric()` Returns True if all characters in the string are numeric

`isdigit()` Returns True if all characters in the string are digits

```
nova = frase.upper()  
print ( nova.islower())
```



`False`

Caracteres ASCII imprimibles

32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	\$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	
61	=	93	]	125	}
62	>	94	^	126	~

El código ASCII es un estándar para la representación de caracteres en cualquier dispositivo electrónico, haciendo referencia a una codificación fija que asigna a caracteres imprimibles como letras, números y signos de puntuación y a caracteres de control no imprimibles un código concreto, que al pasar por el dispositivo lo traduce y muestra el carácter deseado.

Debemos recordar que los ordenadores actúan siempre sobre el sistema binario para realizar sus cálculos, por lo que todas las operaciones hechas en ellos se basan en el uso de ceros y unos.

ASCII se creó bajo la misma premisa definiendo sus caracteres en siete bits, es decir, siete posiciones que muestran un 0 o un 1 y un octavo bit que se reserva para hacer análisis.

Las versiones ampliadas basadas en el American Standard Code for Information Interchange emplean dicho bit para incrementar los caracteres existentes a 256.

TABLE OF SPECIAL CHARACTERS - UNICODE & ISO-8859-1

Char	Code	Char	Code	Char	Code	Char	Code	Char	Code	Char	Code	Char	Code	Char	Code
	160	¡	161	¢	162	£	163	¤	164	¥	165		166	§	167
¨	168	©	169	ª	170	«	171	¬	172		173	®	174	¯	175
°	176	±	177	²	178	³	179	´	180	µ	181	¶	182	·	183
,	184	¹	185	º	186	»	187	¼	188	½	189	¾	190	¿	191
À	192	-	193	Â	194	Ã	195	Ä	196	Å	197	Æ	198	Ç	199
È	200	É	201	Ê	202	Ë	203	Ì	204	Í	205	Î	206	Ï	207
Ð	208	Ñ	209	Ò	210	Ó	211	Ô	212	Õ	213	Ö	214	×	215
Ø	216	Ù	217	Ú	218	Û	219	Ü	220	Ý	221	Þ	222	ß	223
à	224	á	225	â	226	ã	227	ä	228	å	229	æ	230	ç	231
è	232	é	233	ê	234	ë	235	ì	236	í	237	î	238	ï	239
ð	240	ñ	241	ò	242	ó	243	ô	244	õ	245	ö	246	÷	247
ø	248	ù	249	ú	250	û	251	ü	252	ý	253	þ	254	ÿ	255

Unicode es el estándar de codificación de **caracteres universal** utilizado para la representación de texto para procesamiento del equipo. ...

Cada carácter **Unicode** se asigna a un punto de código, que es un número entero entre 0 y 1114111.

`ord ("A")`

65

`ord ("ç")`

231

`chr (231)`

'ç'

P04.1 Quin és el resultat d'aquestes operacions:

a) Passa a majúscules i imprimeix

```
txt = "Castaña, Asunción, María, José"
```

b) Prova aquestes instruccions per veure com treure els accents de cop.

```
mytable = "".maketrans("ÁÉÍÓÚ", "AEIOU");  
txt = "CATALÁN DE CANCIÓN TULÚN!";  
print(txt.translate(mytable));
```

c) Continua el programa anterior passant a majúscules i treient els accents d'aquesta cadena

```
txt2 = "MANÀ manà TITI-RIRI manà manà titi-rirú";
```

P04.2 Quin és el resultat d'aquestes operacions:

c) Estudia aquestes instruccions.

Com es calcula una majúscula a partir d'una minúscula ?

```
print(ord("A"), ord("a"))  
print(ord("B"), ord("b"))  
print(ord("C"), ord("c"))  
print(ord("Ñ"), ord("ñ"))  
print(ord("Á"), ord("á"))  
print(ord("É"), ord("é"))
```

d) Calcula

```
txt = "excursiones, marrones, drones"  
x = txt.replace("es", "s")  
print(x)
```

Caracteres ASCII imprimibles					
32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	\$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

P05.1. Pràctica de demanar dades (pseudocodi)

INICI

// Demana les dades del client

Escriviu "Introdueix el nom:"

Llegir nom

Escriviu "Introdueix el primer cognom:"

Llegir primer_cognom

Escriviu "Introdueix el segon cognom:"

Llegir segon_cognom

// Format del nom (majúscula inicial, resta en minúscules)

nom ← MajúsculaInicial(NomEnMinúscules(nom))

// Format dels cognoms (tot en majúscules)

primer_cognom ← A_MAJUSCULES(primer_cognom)

segon_cognom ← A_MAJUSCULES(segon_cognom)

// Mostra el missatge formatat

Escriviu "Benvingut ", primer_cognom, " ", segon_cognom, " ", nom, "!"

Copilot ens ha donat aquest pseudocodi, que continua a la diapo següent.

Què milloraries tenint en compte el que hem estudiat fins ara ?

En aquest pseudocodi:

NomEnMinúscules(nom) transforma tot el nom en minúscules.

MajúsculaInicial(nom) posa en majúscula la primera lletra.

A_MAJUSCULES(text) transforma tot el text en majúscules.

P05.2. Pràctica de demanar dades (programació)

Tenim el següent enunciat que volem comparar amb el pseudocodi anterior ...

*Demana per separat les dades d'un client: **nom**, **primer_cognom**, **segon_cognom**.*

*Mostra el missatge : Benvingut **primer_cognom segon_cognom**, **nom**.
canviant el nom a minúscules (excepte la primera lletra),
i els cognoms a majúscules*

Que et resulta millor per programar ? Fes el programa.

P06 (avançat). Amplia el programa anterior afegint funcionalitat.

Mostra un missatge que digui :

clau per la base de dades : {clau} on la clau es calcula :

- agafant 4 posicions del cada camp (primer, segon, nom),
- ajunta'ls amb un guió
- i passant tot a majúscules i sense accents.

P07. Entra a la web de w3schools I prova 5 operacions de cadena en el teu quadern.

https://www.w3schools.com/python/python_ref_string.asp

P8. Avançat : Si tenim aquest programa per encriptar paraules de 4 lletres. Fes el programa necessari per desencriptar.

```
cad = input("Entri una paraula de 4 lletres :")
c = ""
c += chr(ord(cad[0])+1)
c += chr(ord(cad[1])+10)
c += chr(ord(cad[2])+8)
c += chr(ord(cad[3])+4)
print ("Resultat encriptat : ", c)
```

```
Entri una paraula de 4 lletres :pero
Resultat encriptat :  qozs
```

- Tenim el valor encriptat = **dyue** A quina paraula correspon ?
- Si encriptem la paraula **cuca** , què passa quan la desencriptem ?

Caracteres ASCII imprimibles					
32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	\$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

P09. Entra a la web de recursos Python, per comprendre la sintaxi de les funcions i practica-les al teu quadern Jupyter

<https://recursospython.com/guias-y-manuales/30-metodos-de-las-cadenas/>

P10. Fes algun exercici d'aquesta web.

<https://aprendeconalf.es/docencia/python/ejercicios/tipos-datos/>

P11. Repàs

- Si ho vols fer en anglès prova w3schools

<https://www.w3schools.com/python/>

Python Tutorial

Python HOME

Python Intro

Python Get Started

Python Syntax

Python Comments

Python Variables ▼

Python Data Types

Python Numbers

Python Casting

Python Strings ▼

Python Booleans

Python Operators

Algunes solucions

#P04

a. CASTAÑA, ASUNCIÓN, MARÍA, JOSÉ

Min Maj

65 97

66 98

b. CATALAN DE CANCION TULUN!

67 99

c. Sumant 32



209 241

193 225

d. excursions, televisions, macarrons

201 233